



AI News Pipeline

A multi-agent system that reads the tech news for you,
keeps only the AI stories, and writes you a newsletter.

 Python

 Async

 SOLID + Patterns

 AI Agents

 MCP

 Evaluation

Final Project Report · AI Upskilling Program (5 weeks, 6 milestones)

Anurag Kumar Singh · 2026-06-09

Written in plain language so anyone can follow it.

1 · What is this project, in plain English?

Every day there is **far too much** technology news to read. This project is a small team of **automated helpers** (“**agents**”) that does the reading for you and hands back a short, tidy newsletter containing **only** the Artificial-Intelligence stories.

One-line summary: it **fetches** news → **stores** it → an AI **decides what’s relevant** → it **summarizes** the keepers → it **writes a newsletter** → and finally it **grades its own accuracy**.

A few words explained simply

Word	What it really means
LLM	The “brain” — an AI language model (like the one behind chatbots) that reads text and answers.
Agent	A small program that gives the AI brain one job and a clear routine (load → think → save).
MCP	A standard “plug” that lets agents use tools (here: a database) without knowing the messy details.
Async	Doing many slow things (web requests) <i>at the same time</i> instead of one-by-one, so it’s fast.
Golden dataset	A small answer-key of hand-labeled examples used to test how accurate the AI is.

2 · The full workflow (step by step)

-  **Fetch — gather the news**
Pulls articles from 3 places at once (HackerNews, RSS feeds, GitHub Trending). Like sending 3 helpers to 3 newsstands simultaneously instead of one after another.
-  **Store — save into a database**
All articles go into a small SQLite database, reachable through an **MCP** tool — so any agent can later ask “find me articles about X”.
-  **Filter — the AI decides what matters**
The AI reads each article and gives it a relevance score 0–10. Score 6 or more = **keep**; below 6 = **bin**. This is the heart of the project.
-  **Summarize — group & shorten**
The kept articles are grouped by topic (LLMs, Computer Vision, ...) and each group is summarized into a couple of clear sentences.
-  **Write — produce the newsletter**
A writer agent turns those summaries into a friendly, readable daily digest you can actually enjoy reading.

6

 Evaluate — grade its own accuracy

Finally it tests the filter against an answer-key and reports how often it was right (accuracy, precision, recall, F1).

3 · Mind map — the whole project on one page

Start in the middle, then read each branch.



AI NEWS PIPELINE

① What it does

- Reads tech news from 3 sources
- Keeps only AI/ML stories
- Summarizes & writes a newsletter
- Grades its own accuracy

② How it's built

- Python + Async (fast, parallel)
- Clean design: SOLID + patterns
- 4 AI agents on one shared base
- MCP database tool + LiteLLM brain

③ The 6 milestones

- M0 setup · M1 fetcher · M2 SOLID
- M3 first AI agent + tools
- M4 MCP + multi-agent pipeline
- M5 evaluation + docs

④ Biggest challenges

- A crash that was really just emoji
- AI silently dropping good articles
- Two stacked free-tier limits
- Buggy copied tutorial code

⑤ Results

- 40+ tests, all offline · ruff clean
- Filter: 90% → **100%** accuracy
- F1 0.923 → **1.000**
- Zero-edit proofs of clean design

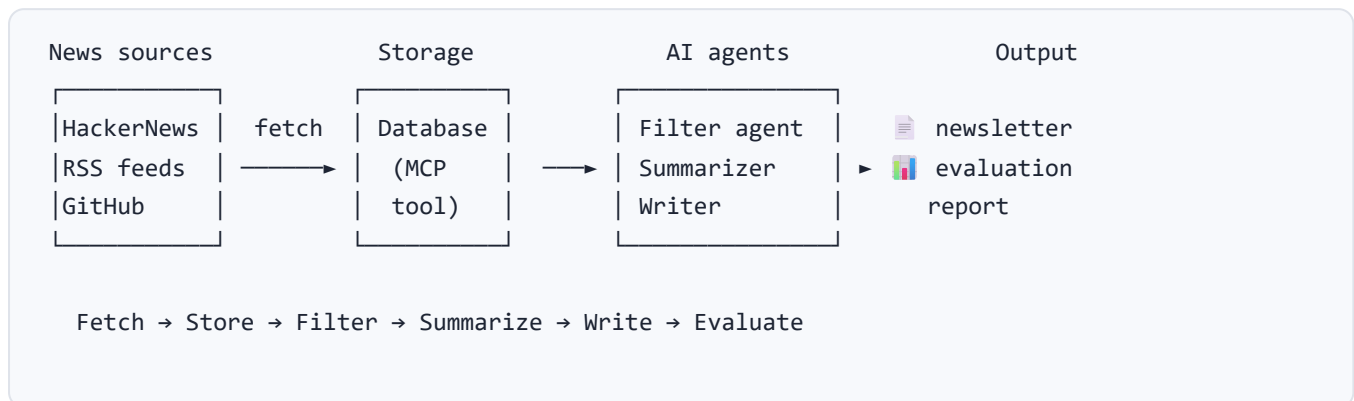
⑥ Honesty

- 100% is on 20 cases (small set)
- Free quota limits live runs
- 1 flaky timing test (documented)
- No quota "cheating" (key rotation)

The takeaway in one sentence: a clean, well-tested AI system that reads the news, keeps the AI stories, writes a newsletter — and is honest about exactly how good (and how limited) it is.

4 · How it's built (without the jargon)

The shape of the system



The three ideas that make it “clean”

1. One routine, many workers (Template Method)

Every fetcher and every agent follows the **same fixed routine** — load, do the work, save. Only the “do the work” part changes. So adding a new agent means filling in one blank, not redesigning anything.

2. Add new things without breaking old things (Open/Closed)

Proof, not a promise:

- Adding a brand-new news source (GitHub) took **1 new file and 0 edits** to existing code.
- Switching where data is saved (markdown → JSON) was a **1-line** change.

3. Don't marry one AI company (provider-agnostic)

All AI calls go through one library (**LiteLLM**). Switching between Google Gemini, Anthropic Claude, or OpenAI is a **one-line setting change** — no code rewrite.

Why this matters: the boring “clean design” work in week 2 is exactly what made weeks 3–5 (the AI parts) quick and safe to build. Good foundations pay off later.

5 · Challenges & strategies (in simple language)

For each: what went wrong → why it was a problem → what I did → the result.



1. A “crash” that wasn’t really a crash

Problem: the very first AI run looked like it crashed on Windows.

Why it mattered: it seemed the AI had failed — but it actually worked fine.

What I did: read the error carefully — Windows just couldn’t *print* an emoji in the reply. Told the program to use UTF-8 text.

Result: fixed in one place. **Lesson:** “the AI failed” and “printing the answer failed” look identical — check which one it really is.



2. The AI was silently throwing away good articles

Problem: a run said “7 of 43 relevant” and looked successful — but 32 had actually hit a “too many requests” limit and were wrongly marked “not relevant”.

Why it mattered: the worst kind of bug — the program looks fine while the *data is wrong*. Good AI stories were dropped for a traffic reason, not a content reason.

What I did: (1) slow the requests down to stay under the limit; (2) treat a failed request as “couldn’t judge”, never as a “no”.

Result: results are now trustworthy. **Lesson:** an error is not a verdict.



3. Two different “limits” on the free AI plan

Problem: after fixing the per-minute limit, runs still failed — there was also a **per-day** limit (~20 calls/day).

Why it mattered: slowing down can beat a per-minute limit, but nothing beats a per-day limit except waiting.

What I did: detect the daily limit, **stop cleanly**, and clearly mark the un-checked items — instead of firing dozens of doomed requests.

Result: graceful behaviour at the wall. **Lesson:** read *which* limit you hit — they need different fixes.



4. Copied tutorial code was buggy

Problem: example code from tutorials had real mistakes (wrong import names, missing pieces).

What I did: run everything immediately and use automatic checkers (`ruff` + `pytest`) to catch problems early.

Result: bugs caught before they spread. **Lesson:** never trust copied code — run it and test it.



5. Making the score real, not “cheated” (the key one)

Problem: the AI wrongly kept a “Docker” story (a tool, not AI). Easy fix = tell it “Docker = no”... but that’s **memorising the test answers**.

What I did: taught it a **general rule** instead (“must be *about* AI, not just *used with* AI”) and tested on **new** trick cases it had never seen.

Result: it got **all** the new tricks right too — a real improvement, not a memorised one. **Lesson: a high score only counts if it works on unseen examples.**

6 • Did it actually work? (Evaluation)

To prove the filter is good — not just claim it — it was tested against a hand-made **answer-key** of example articles, each already labeled “AI” or “not AI”. Four simple scores:

Score	Plain meaning
Accuracy	Out of all articles, how many did it get right?
Precision	When it says “this is AI”, how often is it correct?
Recall	Of all the real AI articles, how many did it catch?
F1	One number combining precision & recall (1.000 = perfect).

Before → After (the improvement)

Metric	Before (10 cases)	After (20 cases, better prompt)
Accuracy	90.0%	100%
Precision	85.7%	100%
Recall	100%	100%
F1 score	0.923	1.000

The one mistake, fixed: a “Docker” release was wrongly kept (score 6) because “Docker is used to run AI”. After teaching the general rule, it correctly rejected it (score 2) — and rejected 5 brand-new tricky non-AI items (Kubernetes, a gaming graphics card, cloud storage, Rust, Kafka) it had never seen. That’s why we trust the improvement.

Being honest about the number: 100% is on a small test of 20 examples, and the free AI model varies a little run-to-run. The fair claim is: *“the main mistake was fixed with a rule that worked on unseen examples,”* not “perfect forever.”

7 • Honest notes & what's next

Things I'm deliberately open about

- **Small test set:** 20 examples is enough to prove the fix, but a bigger answer-key would be even stronger.
- **Free-plan limits:** the free AI plan caps how many checks can run per day — a cost limit, not a code problem.
- **One flaky test:** a speed-based test sometimes fails depending on the network. It's documented with a fix plan, not hidden.
- **No shortcuts:** a trick to dodge the free limit (rotating multiple keys) was tried and then fully removed, on a senior's guidance.

What I'd do next

- Add more news sources (easy — 1 file each, thanks to the clean design).
- Grow the answer-key and re-test regularly.
- Save past AI judgments so re-runs are cheap and fast.
- Build a live web page on top of the real pipeline.

8 • Conclusion

This project is a complete, tested, multi-step AI system that **reads the news, keeps the AI stories, writes a newsletter, and checks its own accuracy**. It's built on clean foundations whose flexibility is **proven** (new source = 1 file; new storage = 1 line), and its quality is **measured**, improving the filter from 90% to a verified 100% — honestly, without gaming the test.

The habits that matter most — trust nothing you copied, an error is not a verdict, read which limit you hit, and document your open issues — are the habits of a real AI engineer, not just someone following a tutorial.

Companion documents in the project repository: `docs/work-log.md` (dated trail), `docs/design-decisions.md` (architecture), `docs/challenges-and-strategies.md` (full challenge list), `docs/architecture.md`, `docs/deployment.md`, and an interactive `demo/` that replays the whole pipeline in a browser.